

Tugas 13: Praktikum & Praktikum Mandiri 13

Pandu Linggar Kumara - 0110221277,
Link GitHub - https://github.com/PanduLgg/M_Learning.git

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: pandulinggar1@gmail.com

Abstract.

Praktikum ini bertujuan untuk memahami dan mengimplementasikan konsep Artificial Neural Network (ANN) melalui dua jenis dataset yang berbeda, yaitu dataset tabular Titanic dan dataset citra Handwritten Digits (MNIST). Pada praktikum pertama, ANN diterapkan untuk menyelesaikan permasalahan klasifikasi biner dengan memprediksi status keselamatan penumpang Titanic berdasarkan fitur demografis dan sosial ekonomi. Proses yang dilakukan meliputi data cleaning, penanganan missing value, encoding data kategorikal, normalisasi fitur, pemisahan data latih dan data uji, serta pembangunan model ANN dengan dua hidden layer dan teknik Early Stopping untuk mencegah overfitting. Pada praktikum kedua, ANN diterapkan pada dataset Handwritten Digits yang berupa data citra angka tulisan tangan. Data citra terlebih dahulu diubah menjadi bentuk numerik melalui normalisasi nilai piksel dan transformasi flatten agar dapat diproses oleh model Multi-Layer Perceptron (MLP). Dataset kemudian dibagi menjadi data latih dan data uji sebelum dilakukan pelatihan model. Arsitektur ANN yang digunakan disesuaikan dengan karakteristik data citra, terutama pada jumlah neuron input dan output layer untuk klasifikasi multi-kelas. Hasil praktikum menunjukkan bahwa ANN mampu memberikan performa yang baik pada kedua jenis dataset, dengan tingkat akurasi yang cukup tinggi. Hal ini membuktikan fleksibilitas ANN dalam menangani berbagai tipe data, baik tabular maupun citra.

1. PRAKTIKUM 13

1.1 Import Library dan Model

Sebelum melakukan analisis dan membangun model Neural Network, kita perlu memanggil pustaka (library) Python yang akan digunakan dalam proses data preprocessing, visualisasi, dan modeling.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import RobustScaler
from sklearn.model_selection import train_test_split

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
```

Gambar 1.1. Import Library & Model

1.2 Memanggil Data set dari Gdrive dan Membaca file .CSV menggunakan Pandas

Sel Dataset yang digunakan pada praktikum ini adalah Titanic Dataset, yang berisi data demografis dan sosial ekonomi penumpang kapal Titanic. Dataset dimuat menggunakan fungsi `pd.read_csv()` dari pustakaPandas

```
from google.colab import drive
drive.mount('/content/drive')
df = pd.read_csv('/content/drive/MyDrive/praktikum ml/praktikum13/data/titanic.csv')
df.head()
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Next steps: [Generate code with df](#) [New interactive sheet](#)

Gambar 1.2. Proses ini hanya perlu dilakukan satu kali per sesi.

1.3 Pemeriksaan Missing Values

Analisis pemeriksaan nilai kosong dilakukan menggunakan fungsi `df.isnull().sum()`. Dari hasil analisis, ditemukan tiga kolom yang memiliki nilai kosong

```
df.isnull().sum()
```

```
***
PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age          177
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin        687
Embarked        2
dtype: int64
```

Gambar 1.3. Missing Values

1.4 Data Cleaning

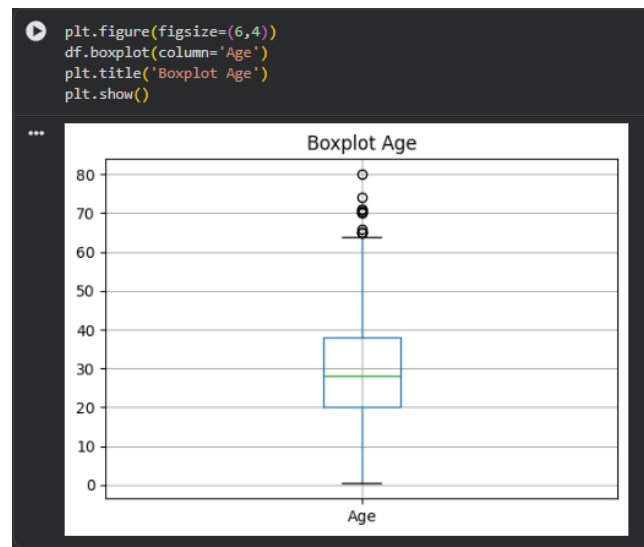
Fungsi `df.drop()` digunakan untuk menghapus kolom yang dianggap kurang relevan atau memiliki terlalu banyak nilai kosong

```
df.drop(['Cabin', 'Name', 'Ticket'], axis=1, inplace=True)
```

Gambar 1.4. drop data

1.5 Visualisasi Distribusi dan Outlier pada Kolom Age

Visualisasi ini membantu memahami karakteristik data umur dan menjadi pertimbangan dalam memilih metode pengisian nilai kosong



Gambar 1.5. Visualisasi Data Distribusi dan Outlier Age

1.6 Mengisi Nilai Kosong pada Kolom Age dengan Median

`df['Age'].median()` menghitung nilai median (nilai tengah) dari kolom Age. `fillna(..., inplace=True)` menggantikan semua nilai kosong pada Age dengan nilai median tersebut langsung di DataFrame `df`. Penggunaan median dipilih karena:

- Lebih robust terhadap outlier.
- Mewakili pusat distribusi umur dengan lebih baik pada data yang tidak simetris sempurna

```
df['Age'].fillna(df['Age'].median(), inplace=True)
```

... /tmp/ipython-input-1933487976.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series, and inplace mode will not be used. The behavior will change in pandas 3.0. This inplace method will never work because the data is a copy. For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method(value, col)' instead.

```
df['Age'].fillna(df['Age'].median(), inplace=True)
```

Gambar 1.6. Isidata Kosong

1.7 Mengisi Nilai Kosong pada Kolom Embarked dengan Modus

`df['Embarked'].mode()[0]` mengambil nilai yang paling sering muncul (modus) pada kolom Embarked. Nilai modus ini kemudian digunakan untuk menggantikan seluruh nilai kosong di kolom tersebut. Karena jumlah missing value pada Embarked sangat sedikit (hanya 2 data), pengisian dengan modus tidak akan mengganggu distribusi data secara signifikan.

```
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)

/tmp/ipython-input-3744086084.py:1: FutureWarning: A value is trying to be
The behavior will change in pandas 3.0. This inplace method will never work

For example, when doing 'df[col].method(value, inplace=True)', try using '

df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
```

Gambar 1.7. Isi data Modus

1.8 Verifikasi Ulang Missing Values

Setelah proses penghapusan kolom dan pengisian nilai kosong, dilakukan pengecekan ulang

```
df.isnull().sum()

0
PassengerId 0
Survived 0
Pclass 0
Sex 0
Age 0
SibSp 0
Parch 0
Fare 0
Embarked 0

dtype: int64
```

Gambar 1.8. Cek ulang Missing Values

1.9 Encoding Data Kategorikal

Tahap ini bertujuan untuk mengubah fitur yang memiliki nilai teks (kategori) menjadi bentuk numerik agar dapat diproses oleh model Neural Network.

```
replacements = {
    'Sex': {'male': 0, 'female': 1},
    'Embarked': {'S': 0, 'C': 1, 'Q': 2}
}

df.replace(replacements, inplace=True)
df.head()
```

```
... /tmp/ipython-input-943814926.py:6: FutureWarning: Downcasting behavior in 'replace'
df.replace(replacements, inplace=True)

  PassengerId  Survived  Pclass  Sex  Age  SibSp  Parch  Fare  Embarked
0           1         0       3    0  22.0     1     0   7.2500         0
1           2         1       1    1  38.0     1     0  71.2833         1
2           3         1       3    1  26.0     0     0   7.9250         0
3           4         1       1    1  35.0     1     0  53.1000         0
4           5         0       3    0  35.0     0     0   8.0500         0
```

Gambar 1.9. Encoding Data

1.10 Split Data

Tahap ini bertujuan untuk memisahkan fitur (X) dan label (y) dari dataset, serta menormalkan skala fitur numerik agar semua kolom memiliki kontribusi yang seimbang ketika diproses oleh model Neural Network.

```
17] X = df.drop('Survived', axis=1)
0s y = df['Survived']
```

Gambar 1.10. Split Data X dan Y

1.11 Transformasi (Scaling)

Tahap ini bertujuan untuk memisahkan fitur (X) dan label (y) dari dataset, serta menormalkan skala fitur numerik agar semua kolom memiliki kontribusi yang seimbang ketika diproses oleh model Neural Network..

```
scaler = RobustScaler()
X_scaled = scaler.fit_transform(X)
```

Gambar 1.11. Transformasi Data

1.12 Train- Test dan Modelling

Sebelum membangun arsitektur model Artificial Neural Network (ANN), langkah pertama yang harus dilakukan adalah memastikan jumlah fitur (kolom) pada data pelatihan.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

X_train.shape
X_train.shape[1]
```

8

Gambar 1.12. Train & Test, Data Modelling

1.13 Model Neural Network (Arsitektur ANN)

Tahap ini merupakan proses membangun model jaringan syaraf tiruan (Artificial Neural Network / ANN) menggunakan library TensorFlow Keras. Model ANN dibangun dengan beberapa lapisan (layer) yang terdiri atas input layer, hidden layer, dan output layer.

```
model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    Dropout(0.3),
    Dense(16, activation='relu'),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: Us
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Gambar 1.13. Membuat Model ANN

1.14 Kompilasi Model

Setelah model Artificial Neural Network (ANN) selesai dibangun, langkah berikutnya adalah melakukan kompilasi model

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

*** Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 32)	288
dropout_4 (Dropout)	(None, 32)	0
dense_7 (Dense)	(None, 16)	528
dropout_5 (Dropout)	(None, 16)	0
dense_8 (Dense)	(None, 1)	17

Total params: 833 (3.25 KB)
Trainable params: 833 (3.25 KB)
Non-trainable params: 0 (0.00 B)

Gambar 1.14. Kompilasi dan Summary

1.15 Early Stopping

Early Stopping adalah salah satu teknik regularization pada Neural Network yang digunakan untuk menghentikan proses pelatihan secara otomatis ketika performa model di data validasi tidak menunjukkan perbaikan setelah beberapa epoch berturut-turut. Dengan cara ini, model tidak akan terus belajar hingga menghafal data latih (overfitting), melainkan berhenti pada titik optimal.

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)
```

Gambar 1.15. Early Stopping

1.16 Training Model Neural Network

Setelah model dibangun dan dikompilasi, langkah selanjutnya adalah melakukan pelatihan model (training) menggunakan data latih ($X_{\text{train}}, y_{\text{train}}$). Pada tahap ini, model akan mempelajari pola dari data input untuk meminimalkan loss function dan meningkatkan accuracy.

```
history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=16,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)
```

```
... Epoch 33/100 --- 0s 4ms/step - accuracy: 0.8166 - loss: 0.4379 - val_accuracy: 0.8252 - val_loss: 0.4153
Epoch 34/100 --- 0s 5ms/step - accuracy: 0.8008 - loss: 0.4254 - val_accuracy: 0.8392 - val_loss: 0.4119
Epoch 35/100 --- 0s 5ms/step - accuracy: 0.8259 - loss: 0.4521 - val_accuracy: 0.8322 - val_loss: 0.4106
Epoch 36/100 --- 0s 4ms/step - accuracy: 0.8169 - loss: 0.4580 - val_accuracy: 0.8392 - val_loss: 0.4075
Epoch 37/100 --- 0s 4ms/step - accuracy: 0.8028 - loss: 0.4659 - val_accuracy: 0.8392 - val_loss: 0.4062
Epoch 38/100 --- 0s 4ms/step - accuracy: 0.7989 - loss: 0.4484 - val_accuracy: 0.8392 - val_loss: 0.4061
Epoch 39/100 --- 0s 5ms/step - accuracy: 0.8307 - loss: 0.4063 - val_accuracy: 0.8392 - val_loss: 0.4062
Epoch 40/100 --- 0s 5ms/step - accuracy: 0.8122 - loss: 0.4770 - val_accuracy: 0.8322 - val_loss: 0.4085
Epoch 41/100 --- 0s 5ms/step - accuracy: 0.8304 - loss: 0.4431 - val_accuracy: 0.8322 - val_loss: 0.4097
Epoch 42/100 --- 0s 5ms/step - accuracy: 0.8111 - loss: 0.4530 - val_accuracy: 0.8392 - val_loss: 0.4035
Epoch 43/100
```

Gambar 1.16. Training Model ANN

1.17 Evaluasi Model (Testing Data)

Setelah proses pelatihan selesai, langkah berikutnya adalah mengukur performa model pada data uji ($X_{\text{test}}, y_{\text{test}}$). Tahap evaluasi ini dilakukan untuk mengetahui kemampuan generalisasi model terhadap data baru yang tidak digunakan saat training.

```
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)
print("Test Loss:", test_loss)
```

```
... 6/6 --- 0s 6ms/step - accuracy: 0.8125 - loss: 0.4204
Test Accuracy: 0.8100558519363403
Test Loss: 0.42732763290405273
```

Gambar 1.17. Evaluasi Model

1.18 Visualisasi Akurasi dan Loss

Setelah proses pelatihan selesai, langkah terakhir adalah menampilkan grafik akurasi dan loss model untuk melihat performa selama training.

```
#VISUALISASI
plt.figure(figsize=(12,4))

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.legend()
plt.title('Model Accuracy')

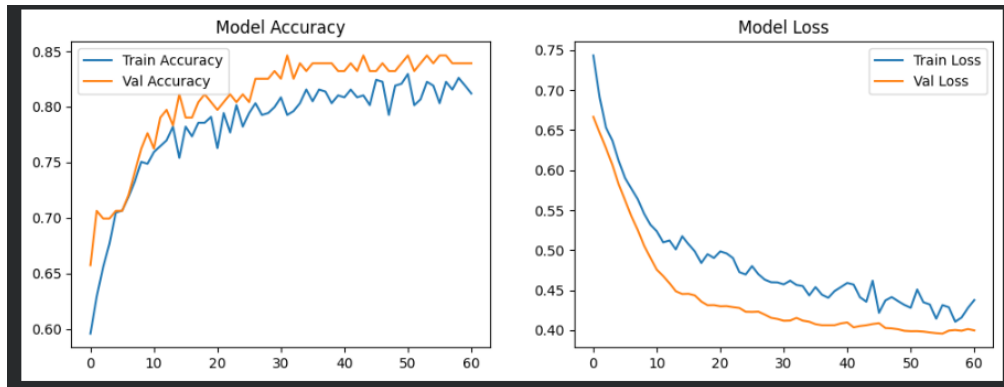
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.legend()
plt.title('Model Loss')

plt.show()
```

Gambar 1.18 Visualisasi Akurasi dan Loss

1.19 Grafik Visualisasi

Visualisasi ini membantu untuk memahami apakah model belajar dengan baik dan apakah terdapat gejala overfitting atau underfitting.



Gambar 1.19 Visualisasi Perbandingan Loss dan Accuracy

2. PRAKTIKUM MANDIRI (MANDIRI)

2.1 Import Library

Bagian ini digunakan untuk memanggil seluruh library yang dibutuhkan selama praktikum. Library NumPy digunakan untuk manipulasi data numerik, Matplotlib untuk visualisasi grafik, Scikit-Learn untuk pembagian data training dan testing, serta TensorFlow Keras untuk membangun dan melatih model Artificial Neural Network (ANN).

```
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.utils import to_categorical

from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.image import load_img, img_to_array
```

Gambar 2.1. Import Library

2.2 Install & Setup Kaggle di Google Colab

Jalankan kode ini di cell Colab:

```
!pip install kaggle

*** Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (1.7.4.5)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.3.0)
Requirement already satisfied: certifi~14.05.14 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2025.11.12)
Requirement already satisfied: charset-normalizer in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.4.4)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.11)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.5)
Requirement already satisfied: python-dateutil>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: setuptools>=21.0.0 in /usr/local/lib/python3.12/dist-packages (from kaggle) (75.2.0)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.17.0)
Requirement already satisfied: text-unidecode in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.1)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.5.1)
```

Gambar 2.2. Install Kaggle

2.3 Load Dataset, Buat folder & pindahkan token

Untuk Menyimpan token di lokasi yang benar dan Mengamankan token, Download dataset dari Kaggle, Simpan kaggle.json, dan Install library kaggle

```
kaggle = '/content/kaggle.json'

mkdir -p ~/.kaggle
cp kaggle.json ~/.kaggle/
chmod 600 ~/.kaggle/kaggle.json

!kaggle datasets download -d olafkrastovski/handwritten-digits-0-9

Dataset URL: https://www.kaggle.com/datasets/olafkrastovski/handwritten-digits-0-9
License(s): CC0-1.0
Downloading handwritten-digits-0-9.zip to /content
 0% 0.00/66.7M [00:00<?, ?B/s]
100% 66.7M/66.7M [00:00<00:00, 1.47GB/s]

from zipfile import ZipFile
import zipfile
import os
file_name = 'handwritten-digits-0-9.zip'

extract_folder = "dataset"
os.makedirs(extract_folder, exist_ok=True)

with zipfile.ZipFile(file_name, 'r') as zip:
    zip.extractall(extract_folder)
    print('done')

done

data_0 = os.listdir('/content/dataset/0')
data_1 = os.listdir('/content/dataset/1')
data_2 = os.listdir('/content/dataset/2')
data_3 = os.listdir('/content/dataset/3')
data_4 = os.listdir('/content/dataset/4')
data_5 = os.listdir('/content/dataset/5')
data_6 = os.listdir('/content/dataset/6')
data_7 = os.listdir('/content/dataset/7')
data_8 = os.listdir('/content/dataset/8')
data_9 = os.listdir('/content/dataset/9')

len(data_0)

2236
```

Gambar 2.3. Dataset Load

2.4 Pemeriksaan Data Awal

Tahap ini digunakan untuk memahami struktur awal dataset, seperti jumlah data, bentuk data, dan ukuran citra. Pemeriksaan ini penting untuk menentukan langkah preprocessing yang sesuai sebelum data digunakan dalam model.

```
(X, y), (X_test_default, y_test_default) = mnist.load_data()

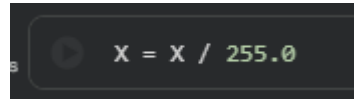
print("Shape data awal :", X.shape)
print("Shape label      :", y.shape)

*** Shape data awal : (60000, 28, 28)
    Shape label      : (60000,)
```

Gambar 2.4. Data Cek

2.5 Normalisasi Pixel

Tahap preprocessing bertujuan untuk menyiapkan data agar siap diproses oleh ANN. Melakukan normalisasi nilai piksel agar berada pada rentang Nilai pixel: 0–255 → 0–1 (numerik terstandarisasi)

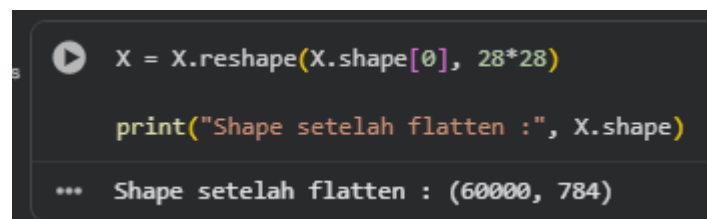


```
X = X / 255.0
```

Gambar 2.5. Normalisai Pixel

2.6 Transformasi Citra ke Bentuk Numerik (Flatten)

Dari (60000, 28, 28) Menjadi (60000, 784) → numerik seperti data tabular

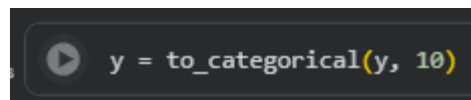


```
X = X.reshape(X.shape[0], 28*28)
print("Shape setelah flatten :", X.shape)
*** Shape setelah flatten : (60000, 784)
```

Gambar 2.6. Transformasi Data

2.7 Encoding Target (Numerik)

Label angka (0–9) → vektor numerik one-hot

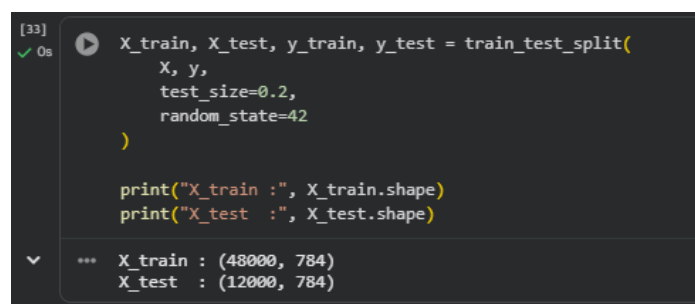


```
y = to_categorical(y, 10)
```

Gambar 2.7. Encoding Data

2.8 Split Data Training dan Testing

Menggantikan split default MNIST

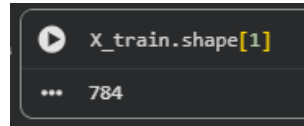


```
[33] ✓ 0s X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
print("X_train :", X_train.shape)
print("X_test  :", X_test.shape)
*** X_train : (48000, 784)
    X_test  : (12000, 784)
```

Gambar 2.8. Split Data Train & Test

2.9 Menentukan Jumlah Fitur (Input Layer)

Digunakan sebagai input_shape ANN, Output: 784



Gambar 2.9. Input Layer

2.10 Pembuatan Model ANN

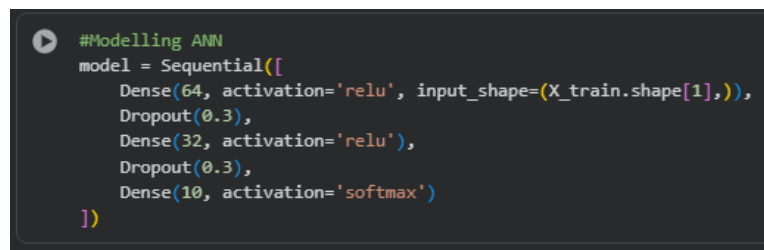
Struktur:

Input layer: 784 neuron

Hidden layer 1: 64 neuron (ReLU)

Hidden layer 2: 32 neuron (ReLU)

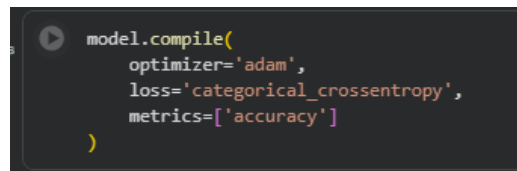
Output layer: 10 neuron (Softmax)



Gambar 2.10. Modelling ANN

2.11 Kompilasi Model

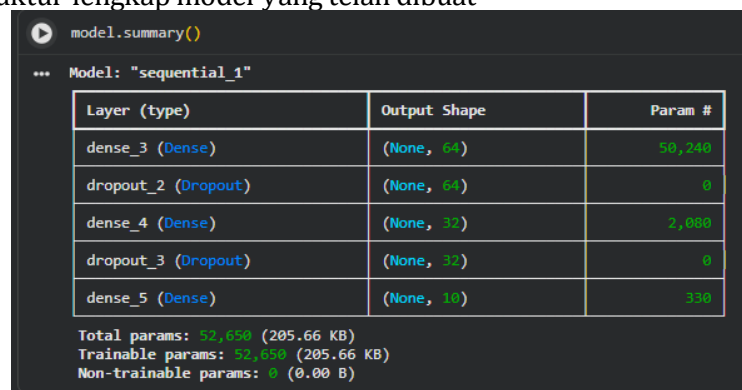
Setelah model Artificial Neural Network (ANN) selesai dibangun, langkah berikutnya adalah melakukan kompilasi model



Gambar 2.11. Kompilasi Model ANN

2.12 Ringkasan Model

Menampilkan struktur lengkap model yang telah dibuat



Gambar 2.12. Model Summary

2.13 Early Stopping

Early Stopping adalah salah satu teknik regularization pada Neural Network yang digunakan untuk menghentikan proses pelatihan secara otomatis ketika performa model di data validasi tidak menunjukkan perbaikan setelah beberapa epoch berturut-turut

```
early_stop = EarlyStopping(  
    monitor='val_loss',  
    patience=5,  
    restore_best_weights=True  
)
```

Gambar 2.13. EarlyStop

2.14 Training Model Neural Network

Setelah model dibangun dan dikompilasi, langkah selanjutnya adalah melakukan pelatihan model (training) menggunakan data latih (X_{train}, y_{train}). Pada tahap ini, model akan mempelajari pola dari data input untuk meminimalkan loss function dan meningkatkan accuracy.

```
history = model.fit(  
    X_train, y_train,  
    epochs=50,  
    batch_size=128,  
    validation_split=0.2,  
    callbacks=[early_stop],  
    verbose=1  
)
```

```
... Epoch 2/50  
300/300 ————— 1s 5ms/step - accuracy: 0.8650 - loss: 0.4650 - val_accuracy: 0.9329 - val_loss: 0.2345  
Epoch 3/50  
300/300 ————— 1s 4ms/step - accuracy: 0.8954 - loss: 0.3587 - val_accuracy: 0.9411 - val_loss: 0.1981  
Epoch 4/50  
300/300 ————— 1s 5ms/step - accuracy: 0.9093 - loss: 0.3154 - val_accuracy: 0.9470 - val_loss: 0.1795  
Epoch 5/50  
300/300 ————— 2s 5ms/step - accuracy: 0.9178 - loss: 0.2874 - val_accuracy: 0.9510 - val_loss: 0.1677  
Epoch 6/50  
300/300 ————— 2s 7ms/step - accuracy: 0.9262 - loss: 0.2576 - val_accuracy: 0.9575 - val_loss: 0.1520  
Epoch 7/50  
300/300 ————— 2s 6ms/step - accuracy: 0.9338 - loss: 0.2346 - val_accuracy: 0.9561 - val_loss: 0.1505  
Epoch 8/50  
300/300 ————— 2s 5ms/step - accuracy: 0.9361 - loss: 0.2218 - val_accuracy: 0.9579 - val_loss: 0.1465  
Epoch 9/50  
300/300 ————— 1s 5ms/step - accuracy: 0.9389 - loss: 0.2117 - val_accuracy: 0.9601 - val_loss: 0.1392  
Epoch 10/50  
300/300 ————— 1s 5ms/step - accuracy: 0.9416 - loss: 0.2041 - val_accuracy: 0.9591 - val_loss: 0.1352  
Epoch 11/50  
300/300 ————— 1s 5ms/step - accuracy: 0.9424 - loss: 0.1997 - val_accuracy: 0.9625 - val_loss: 0.1298  
Epoch 12/50  
300/300 ————— 1s 4ms/step - accuracy: 0.9475 - loss: 0.1868 - val_accuracy: 0.9618 - val_loss: 0.1331
```

Gambar 2.14. Train Model

2.15 Evaluasi Model

Setelah proses pelatihan selesai, langkah berikutnya adalah mengukur performa model pada data uji (X_{test}, y_{test}). Tahap evaluasi ini dilakukan untuk mengetahui kemampuan generalisasi model terhadap data baru yang tidak digunakan saat training. Umumnya akurasi: 95–97%

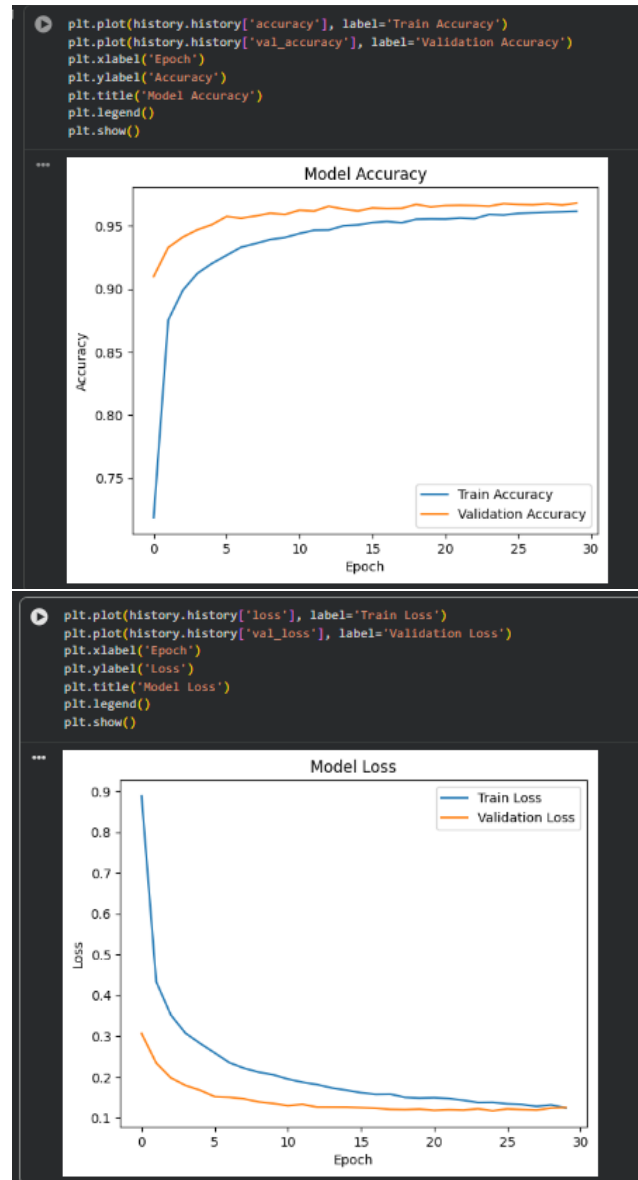
```
test_loss, test_acc = model.evaluate(X_test, y_test)  
  
print("Test Akurasi :", test_acc)  
print("Test Loss    :", test_loss)
```

```
... 375/375 ————— 1s 2ms/step - accuracy: 0.9660 - loss: 0.1131  
Test Akurasi : 0.9665833115577698  
Test Loss    : 0.11352945864200592
```

Gambar 2.15. Evaluasi Model Neural

2.16 Visualisasi Accuracy & Loss

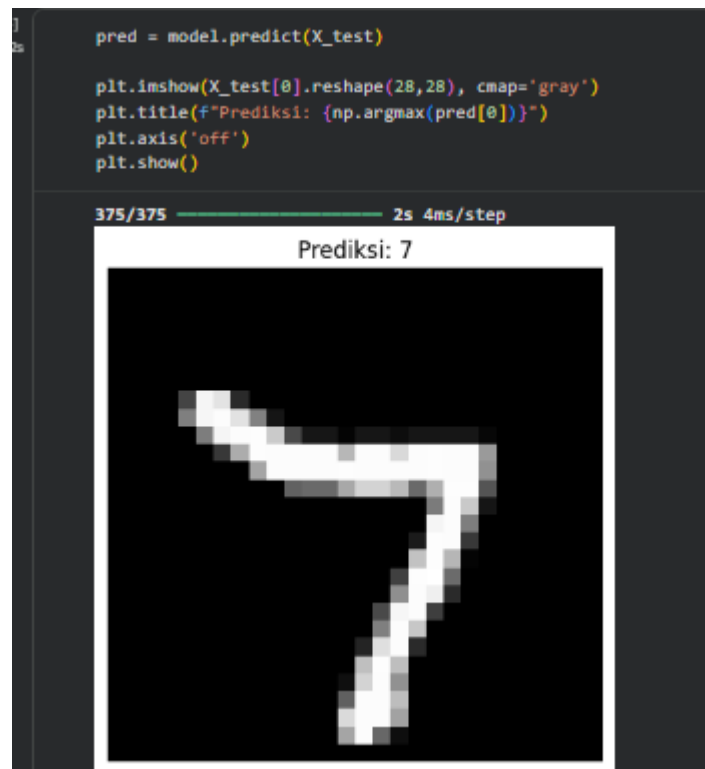
Pada tahap ini, source code digunakan untuk menampilkan grafik performa model selama proses training. Data yang divisualisasikan diambil dari objek history yang dihasilkan oleh fungsi model.fit(). Grafik yang ditampilkan terdiri dari akurasi dan loss pada data training dan data validasi di setiap epoch.



Gambar 2.16. Visualisasi Perbandingan

2.17 Prediksi Contoh

Tahap prediksi contoh digunakan untuk menguji kemampuan model ANN dalam mengenali data baru yang belum pernah digunakan saat training maupun testing. Source code pada tahap ini melakukan prediksi terhadap satu sampel data uji menggunakan fungsi `model.predict()`. Teks judul menunjukkan hasil prediksi angka, Jika prediksi sesuai dengan bentuk angka pada gambar, maka dapat disimpulkan bahwa model berhasil mengenali pola citra dengan baik.



Gambar 2.17. Prediksi Angka Muncul

KESIMPULAN

Berdasarkan hasil pelaksanaan Praktikum 13 Machine Learning yang mencakup penerapan Artificial Neural Network (ANN) pada dataset Titanic dan Handwritten Digits, dapat disimpulkan bahwa ANN merupakan metode pembelajaran mesin yang fleksibel dan efektif untuk menangani berbagai jenis data serta permasalahan klasifikasi. Pada dataset Titanic, ANN digunakan untuk memprediksi status keselamatan penumpang yang bersifat klasifikasi biner. Proses preprocessing memegang peranan penting, khususnya dalam penanganan missing value, penghapusan fitur yang tidak relevan, encoding data kategorikal, serta normalisasi data. Tahapan ini terbukti mampu meningkatkan kualitas data sehingga model dapat belajar secara optimal. Model ANN pada dataset Titanic dibangun menggunakan arsitektur Sequential dengan dua hidden layer dan fungsi aktivasi ReLU, serta output layer dengan fungsi aktivasi Sigmoid. Penggunaan optimizer Adam dan loss function Binary Crossentropy memberikan hasil pelatihan yang stabil, sementara penerapan Early Stopping efektif dalam mencegah terjadinya overfitting. Hasil evaluasi menunjukkan bahwa model mampu mencapai tingkat akurasi yang cukup baik pada data uji, menandakan bahwa model memiliki kemampuan generalisasi yang memadai terhadap data baru.

Pada praktikum kedua, ANN diterapkan pada dataset Handwritten Digits yang memiliki karakteristik berbeda karena berupa data citra. Perbedaan utama terletak pada tahapan preprocessing, di mana data gambar harus terlebih dahulu diubah menjadi bentuk numerik melalui normalisasi nilai piksel dan proses flatten. Transformasi ini memungkinkan data citra diperlakukan seperti data tabular sehingga dapat diproses oleh model Multi-Layer Perceptron (MLP). Selain itu, target output pada dataset ini bersifat multi-kelas, sehingga digunakan fungsi aktivasi Softmax dan loss function Categorical Crossentropy pada output layer. Hasil pelatihan model pada dataset Handwritten Digits menunjukkan tingkat akurasi yang sangat baik, yang menandakan bahwa ANN mampu mengenali pola visual sederhana seperti angka tulisan tangan secara efektif meskipun tanpa menggunakan arsitektur khusus citra seperti Convolutional Neural Network (CNN). Hal ini menegaskan bahwa ANN dasar tetap relevan sebagai pendekatan awal dalam pengenalan pola citra.

Referensi:

scikit-learn Developers

Breast Cancer Dataset — sklearn.datasets

https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html

Kamyar Azar

Breast Cancer Classification Using PCA in Python

<https://www.kaggle.com/code/kamyarazar/breast-cancer-classification-using-pca-in-python>

UCI Machine Learning Repository

Breast Cancer Wisconsin (Diagnostic) Dataset

[https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+\(Diagnostic\)](https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic))